ROS 2 기반 **주차 슬롯 분류 및 사용자 자격 반영형** 자율 주차 시스템

Team 베이맥스

2391007 김민정
2391017 이윤지



목차

01
프로젝트 주제

02
목표 선정

03
시스템 구조와
구현 방법

04
결과 및 데모

05
어려웠던 구간

06
향후 보완점

기존의 자율 주차에 **존재하지 않았던 것이 무엇인가**

기존 프로젝트

어느 칸이 비어 있는가

주차 슬롯 점유 여부 판별 · 장애물 회피 중심

본 프로젝트

누가 그 자리에 댈 수 있는가

빈 슬롯이 사용자 자격과 호환되는지까지 판단

국내의 전용주차구역 종류는 자격과 조건에 따라서 다양하게 분화하였음

문제 설정의 단순화를 위해 여러 자격 유형 중 **국제 규격(ISA)이** 존재하는 **장애인 전용** 슬롯으로 범위를 한정



여성을 위한 여성 전용 주차구역



경차 보급을 위한 경차 전용 주차구역



보행성 장애인을 위한 장애인 전용 주차구역



보행약자인 노인을 위한 어르신 우선 주차구역



전기차 충전을 위한 전기차 전용 주차구역

02 목표 설정

단일 파이프라인으로 연결

시뮬레이션 환경 구성

Gazebo Classic에서 자아 차량, 주차 차량, 주차 슬롯, 방지턱을 배치하고 /ego/odom, /ego/scan, 카메라 이미지 토픽을 발행한다.

자율 주행 구간: Nav2

parking_node는 Nav2의 NavigateToPose action을 사용해 차량을 주차장 진입점, 관측 지점, 목표 슬롯 앞 레인 위치까지 이동시킨다.
이 구간에서는 Nav2가 경로 계획과 추종을 담당하고, /cmd_vel 명령은 cmd_vel_relay를 통해 /ego/cmd_vel로 전달된다.

슬롯 관측: Vision Node + YOLOv8

관측 지점에 도착하면 parking_node가 /parking/obs_trigger를 발행한다.
vision_node는 좌우 카메라, LiDAR, 슬롯 메타데이터를 이용해 슬롯 상태를 판단한다.
YOLOv8 모델이 로드된 경우 카메라 이미지에서 차량을 감지해 주차 슬롯의 점유 여부 판단에 활용한다.

02 목표 설정

단일 파이프라인으로 연결

슬롯 선택: Decision Node

decision_node는 /parking/slot_states를 받아 빈 슬롯을 필터링한다.
일반 사용자는 장애인 전용 슬롯을 제외하고, 장애인 자격 사용자는 장애인 전용 슬롯을 우선 선택한다.
이후 진입점과 가까운 슬롯을 목표 슬롯으로 정해 /parking/target_slot에 발행한다.

정밀 주차: Parking Node 직접 제어

목표 슬롯이 정해지면 parking_node가 Nav2로 슬롯 앞까지 이동한 뒤, 직접 /ego/cmd_vel을 발행한다.
TURN_TO_SLOT에서 방향을 맞추고, X_ALIGN에서 슬롯 중심과 정렬한 뒤, PARK_REVERSE에서 linear.x < 0 명령으로 후진 주차를 수행한다.

03 시스템 구조 및 구현 방법

Gazebo 주차장 시뮬레이션 환경 구성

Gazebo에서 주차장 world와 차량 모델을 실행하고, 차량 센서와 주행 환경을 시뮬레이션한다.

- Gazebo world 파일 + YAML 슬롯 메타데이터 + launch에서 동적 차량 삽입 방식

1. Gazebo world로 물리적 주차장 제작

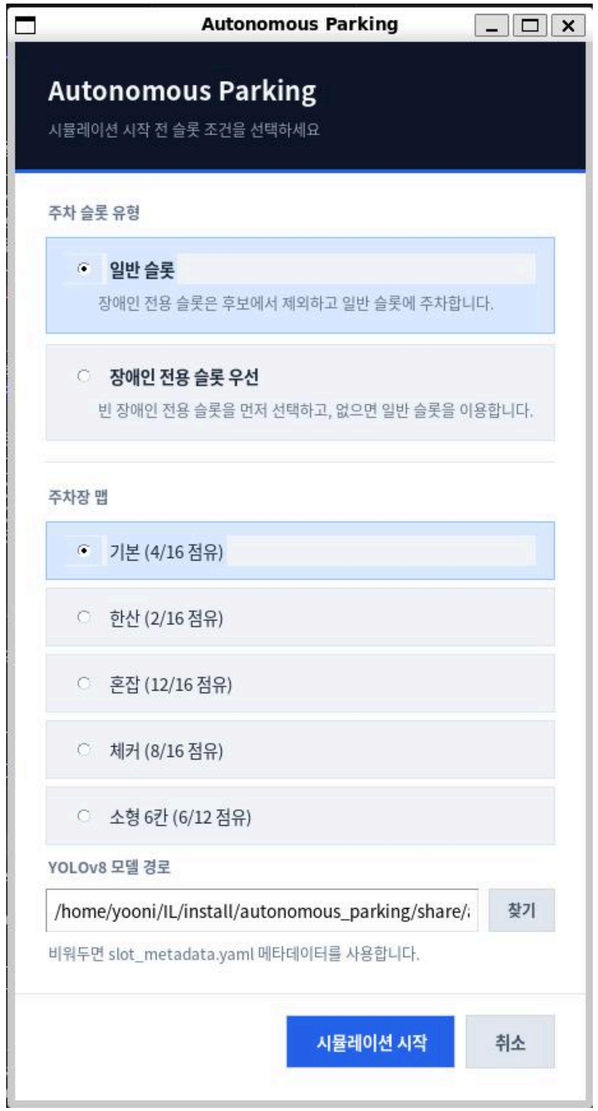
주차장의 실제 배경은 parking_lot.world 내부에 SDF 형식으로 작성

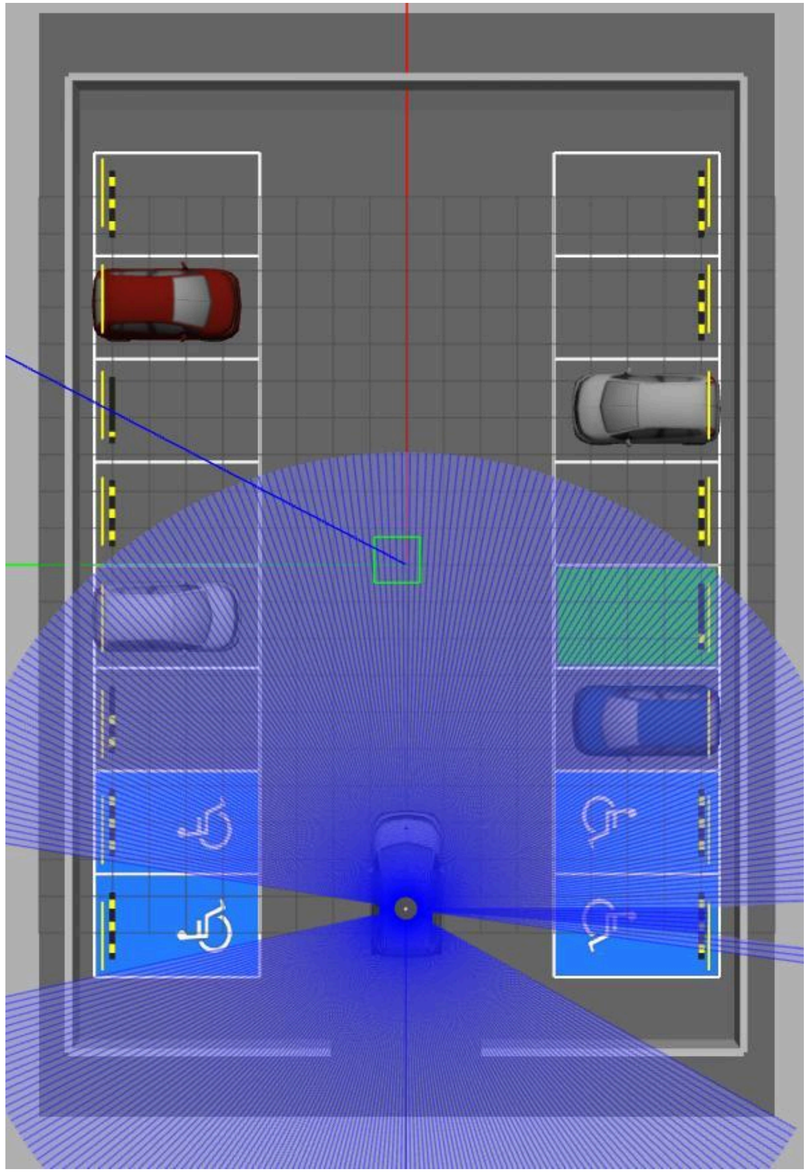
바닥: 30m × 20m 크기의 ground plane
외벽: 북/남/동/서쪽 벽
서쪽 입구: 벽을 위/아래로 나누어 차량 진입 공간 확보
주차선: 흰색 얇은 box 모델로 슬롯 경계 표현
중앙 주행 레인: y=-3.0 ~ y=3.0 범위
북쪽 A열, 남쪽 B열 주차 슬롯 배치

2. YAML로 슬롯 의미 정보 정의

물리적인 선은 world에서, 각 슬롯의 ID, 타입, 좌표, 크기, 점유 여부는 YAML 파일에 따로 정의

주차장 크기: width 22.4m, height 17.0m
중앙 레인: y=-3.0 ~ y=3.0
관측 지점: obs_1, obs_2
차량 시작 위치: x=-12.0, y=0.0
진입 지점: x=-10.5, y=0.0
슬롯 ID: A1A8, B1B8
슬롯 타입: handicapped, general
슬롯 중심 좌표와 ROI
초기 점유 상태





03 시스템 구조 및 구현 방법

Gazebo 주차장 시뮬레이션 환경 구성

Gazebo에서 주차장 world와 차량 모델을 실행하고, 차량 센서와 주행 환경을 시뮬레이션한다.

- Gazebo world 파일 + YAML 슬롯 메타데이터 + launch에서 동적 차량 삽입 방식

3. 여러 주차장 시나리오 구성

맵 설정은 default, sparse, crowded, alternating, compact로 나누었음



4. 점유 차량은 launch에서 동적으로 삽입

처음부터 world 파일에 모든 주차 차량을 고정해둔 것이 아니라,
YAML에서 occupied: true인 슬롯을 읽고 launch 단계에서 해당 위치에 주차 차량 모델을 삽입

[parking_sim.launch.py]
- (line 411) 점유 차량을 world에 동적으로 삽입하는 로직 시작
- (line 417) YAML 슬롯 목록 순회
- (line 418) occupied가 true인 슬롯만 선택
- (line 428) 차량 모델 include XML 생성
- (line 438) 기존 world 파일 읽기
- (line 445) 임시 world 파일 생성 후 Gazebo에 전달

ROS 2 노드 기반 인식·판단·제어 구조 설계

1. 기능을 인식·판단·제어로 분리

자율 주차 전체 기능을 한 노드에 모두 넣지 않고, 역할에 따라 노드를 분리
vision_node: 주차 슬롯 상태 인식
decision_node: 사용자 자격 기반 목표 슬롯 선택
parking_node: 주행 및 주차 제어
cmd_vel_relay: Nav2와 Gazebo 차량 사이의 속도 명령 중계

2. 시뮬레이터 센서 데이터를 ROS 2 토픽으로 입력

Gazebo 차량 모델에서 카메라, LiDAR, odometry 데이터를 발행하고,
각 노드가 필요한 데이터를 토픽으로 구독하도록 구성

- 카메라 이미지: /ego/camera/image_raw, /ego/camera_left/image_raw, /ego/camera_right/image_raw
- LiDAR: /ego/scan
- 차량 위치 및 자세: /ego/odom

ROS 2 노드 기반 인식·판단·제어 구조 설계

3. vision_node에서 슬롯 상태 생성

vision_node는 센서 데이터와 슬롯 메타데이터를 이용해 각 주차 슬롯의 상태를 판단

- 슬롯 메타데이터 YAML 로드
- 카메라 이미지 수신
- YOLOv8 모델이 있으면 차량 감지 수행
- LiDAR 기반 점유 여부 보조 판단
- 슬롯별 ID, 타입, 위치, 점유 여부를 JSON으로 정리
- /parking/slot_states로 발행
- 차선 중심 offset을 /parking/lane_offset으로 발행

4. decision_node에서 목표 슬롯 선택

decision_node는 인식 결과를 받아 사용자의 자격 조건에 맞는 목표 슬롯을 선택

- /parking/slot_states 구독
- 점유된 슬롯 제외
- 일반 사용자는 장애인 전용 슬롯 제외
- 장애인 사용자는 장애인 전용 슬롯 우선 선택
- 진입점과 가까운 슬롯을 최종 목표로 선택
- /parking/target_slot로 목표 슬롯 발행
- 선택 가능한 슬롯이 없으면 /parking/no_slot 발행

ROS 2 노드 기반 인식·판단·제어 구조 설계

5. parking_node에서 상태 머신 기반 제어

parking_node는 선택된 목표 슬롯을 기반으로 차량의 전체 주차 과정을 제어

- 차량 위치 /ego/odom 수신
- 목표 슬롯 /parking/target_slot 수신
- 상태 머신으로 주차 흐름 관리
- 진입, 대기, 레인 주행, 회전, 정렬, 후진 주차 수행
- 정밀 주차 단계에서 /ego/cmd_vel 직접 발행

ROS 2 노드 기반 인식·판단·제어 구조 설계

6. Nav2와 Gazebo 차량 사이의 속도 명령 연결

Nav2는 /cmd_vel로 속도 명령을 발행
Gazebo 차량은 /ego/cmd_vel을 구독함
⇒ 둘의 연결을 위해 사용됨

Nav2 controller_server
→ /cmd_vel
→ cmd_vel_relay
→ /ego/cmd_vel
→ ego_hatchback

7. 토픽 기반 비동기 파이프라인 구성

각 노드는 직접 함수를 호출하지 않고 ROS 2 토픽으로 데이터를 주고 받음

Gazebo Sensors
→ vision_node
→ /parking/slot_states
→ decision_node
→ /parking/target_slot
→ parking_node
→ /ego/cmd_vel
→ ego_hatchback

YOLOv8, LiDAR, 슬롯 메타데이터 기반 슬롯 인식 구현 과정

1. 슬롯 메타데이터 로드

먼저 YAML 파일에서 주차 슬롯 정보를 불러옴

- 슬롯 ID: A1, A2, B1 등
- 슬롯 타입: general, handicapped
- 슬롯 중심 좌표: center_x, center_y
- 슬롯 방향: yaw
- 슬롯 크기: width, depth
- ROI 영역: x_min, x_max, y_min, y_max
- 초기 점유 여부: occupied

2. 센서 데이터 수신

vision_node는 Gazebo 차량에서 발행되는 센서 데이터를 ROS 2 토픽으로 구독

사용 센서:

- 전방 카메라
- 좌측 카메라
- 우측 카메라
- LiDAR
- Odometry

YOLOv8, LiDAR, 슬롯 메타데이터 기반 슬롯 인식 구현 과정

3. YOLOv8 모델 로드

실행 시 YOLOv8 .pt 모델 경로를 파라미터로 받고, 모델 파일이 있으면 로드
→ 모델이 없으면 YAML의 초기 점유 정보나 LiDAR 기반 판단으로 fallback

4. 카메라 이미지 기반 차량 검출

수신한 카메라 이미지를 BGR 배열로 변환한 뒤 YOLOv8에 입력

YOLOv8은 이미지 안의 차량 객체를 검출하고, 검출된 bounding box를 반환

5. ROI 투영 및 슬롯 매칭

검출된 차량이 어느 슬롯에 해당하는지 판단하기 위해 슬롯의 실제 좌표와 ROI 정보를 활용

수행 방식

- 슬롯 메타데이터의 중심 좌표와 ROI 확인
- 차량 현재 위치와 yaw 기준으로 슬롯 위치 해석
- 카메라 검출 결과와 슬롯 후보를 매칭
- 일정 거리 범위 안의 슬롯만 유효 후보로 사용

YOLOv8, LiDAR, 슬롯 메타데이터 기반 슬롯 인식 구현 과정

6. LiDAR 기반 점유 판단 보조

LiDAR 거리 데이터를 사용해 슬롯 방향에 물체가 있는지 확인
YOLOv8 검출이 가능할 때는 LiDAR를 보조 판단으로 사용하고, YOLOv8이 없을 때는 LiDAR가 점유 여부 판단의 주된 기준이 될 수 있다.

7. 점유 판단 결과 평활화

프레임 단위 인식 결과는 흔들릴 수 있으므로, 여러 프레임의 판단 결과를 투표 방식으로 누적해 안정화

8. 슬롯 상태 토픽 발행

최종적으로 각 슬롯의 상태를 JSON 형태로 정리해 /parking/slot_states 토픽으로 발행

포함 정보
- 슬롯 ID
- 슬롯 타입
- 위치
- ROI
- 점유 여부

사용자 자격 기반 목표 슬롯 선택

decision_node에서 담당

vision_node가 발행한 슬롯 상태 정보를 받아서, 현재 사용자의 자격에 맞는 주차 슬롯을 선택하는 과정

1. 사용자 자격 입력

실행 시 user_credential 파라미터로 사용자의 자격을 입력받는다.

- general: 일반 사용자
- handicapped: 장애인 전용 슬롯 이용 가능 사용자

2. 슬롯 상태 수신

decision_node는 /parking/slot_states 토픽을 구독,
이 토픽에는 각 슬롯의 ID, 타입, 위치, 점유 여부가 JSON 형태로 포함됨

- /parking/slot_states 구독
- 슬롯 상태 콜백 함수
- JSON 데이터 파싱
- 슬롯 ID 기준으로 슬롯 상태 저장

3. 점유 슬롯 제외

수신한 슬롯 목록에서 이미 차량이 있는 슬롯은 후보에서 제외

- 슬롯 선택 함수 시작
- 빈 슬롯만 후보로 선택
- occupied가 false인 슬롯만 남김

user_credential 입력

↓

/parking/slot_states 수신

↓

점유 슬롯 제외

↓

사용자 자격별 필터링

↓

후보 없음 → /parking/no_slot 발행

↓

후보 있음 → 우선순위 정렬

↓

/parking/target_slot 발행

사용자 자격 기반 목표 슬롯 선택

decision_node에서 담당

vision_node가 발행한 슬롯 상태 정보를 받아서, 현재 사용자의 자격에 맞는 주차 슬롯을 선택하는 과정

4. 사용자 자격별 필터링

일반 사용자인 경우 장애인 전용 슬롯을 제외하고, general 타입 슬롯만 후보로 남긴다.
장애인 자격 사용자인 경우 장애인 전용 슬롯도 후보에 포함하며, 이후 정렬 단계에서 장애인 슬롯을 우선 선택

5. 선택 가능한 슬롯이 없는 경우 처리

조건을 만족하는 슬롯이 하나도 없으면 /parking/no_slot을 발행
이 신호를 통해 주차 제어 노드는 주차 가능한 슬롯이 없다는 상황을 인식 가능

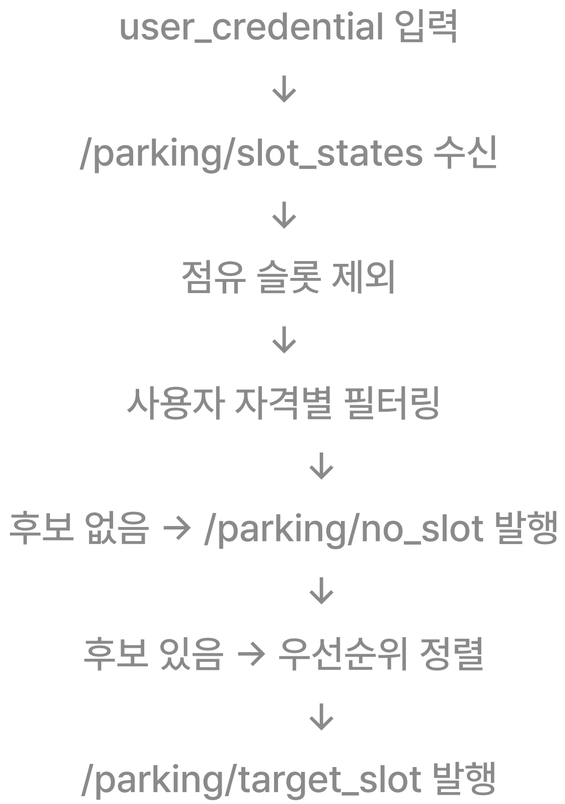
6. 우선순위 기반 슬롯 정렬

후보 슬롯이 있으면 우선순위에 따라 정렬

- 장애인 자격 사용자는 handicapped 타입 슬롯 우선
- 같은 우선순위 안에서는 진입점 x=-10.5와 가까운 슬롯 우선

7. 최종 목표 슬롯 발행

정렬된 후보 중 첫 번째 슬롯을 최종 목표 슬롯으로 선택하고 /parking/target_slot 토픽으로 발행



Nav2 NavigateToPose 기반 자율 주행

1. Nav2 Stack 실행

parking_sim.launch.py에서 Nav2의 핵심 노드들을 실행

- Nav2 stack 실행 구간
- controller_server 실행
- planner_server 실행
- behavior_server 실행
- bt_navigator 실행
- Nav2 lifecycle manager 실행

2. Nav2 파라미터 설정

Nav2가 어떤 방식으로 경로를 계획하고 추종할지 nav2_params.yaml에 설정

- Nav2 navigate_to_pose navigator 설정
- controller goal checker 설정
- 목표 도착 허용 오차 설정

Nav2 NavigateToPose 기반 자율 주행

3. parking_node에서 NavigateToPose ActionClient 생성

parking_node는 Nav2에 목표 지점을 보내기 위해 NavigateToPose ActionClient 생성

- NavigateToPose import
- navigate_to_pose ActionClient 생성
- Nav2 goal 상태 변수 초기화

4. 목표 지점 전송 함수 구현

목표 좌표 (x, y)를 받아 PoseStamped goal을 만들고 Nav2에 비동기로 전송

- _send_nav_goal(x, y) 함수 시작
- Nav2 action server 준비 여부 확인
- NavigateToPose.Goal() 생성
- goal frame을 odom으로 설정
- 목표 x좌표 설정
- 목표 y좌표 설정
- Nav2 goal 비동기 전송

Nav2 NavigateToPose 기반 자율 주행

5. Goal 응답 및 결과 처리

Nav2가 goal을 수락했는지 확인하고, 주행 완료 결과를 받아 성공/실패 상태를 저장

- goal 응답 콜백
- goal handle 수신
- goal reject 처리
- 결과 콜백 등록
- Nav2 result 콜백
- 성공 여부 저장

6. Nav2 속도 명령을 차량에 전달

Nav2의 controller_server는 /cmd_vel을 발행하지만, Gazebo 차량은 /ego/cmd_vel을 사용하므로 cmd_vel_relay가 중계

- cmd_vel_relay 실행
- /ego/cmd_vel publisher 생성
- /cmd_vel subscriber 생성
- 받은 속도 명령을 그대로 재발행

parking_node

→ NavigateToPose goal 생성

→ navigate_to_pose action server로 전송

→ Nav2 planner/controller가 경로 계획 및 추종

→ /cmd_vel 발행

→ cmd_vel_relay

→ /ego/cmd_vel

→ ego_hatchback 이동

직접 /ego/cmd_vel 제어 기반 정밀 후진 주차

목표 슬롯 앞까지 접근한 뒤, Nav2가 아니라 parking_node가 직접 차량 속도 명령을 발행해 주차를 마무리하는 과정
정밀한 회전, 정렬, 후진 동작이 필요하기 때문에 /ego/cmd_vel을 직접 제어

1. 목표 슬롯 정보 설정

decision_node가 선택한 목표 슬롯을 parking_node가 수신하면, 해당 슬롯의 위치와 방향을 기준으로 주차 목표를 설정

- 목표 슬롯 ID 확인
- 슬롯 중심 좌표 center_x, center_y 저장
- 슬롯이 A열인지 B열인지 확인
- A열은 남쪽을 바라본 뒤 후진
- B열은 북쪽을 바라본 뒤 후진
- 슬롯 방향에 따라 목표 yaw 설정

2. 슬롯 앞 레인 위치까지 이동

차량은 바로 슬롯 안으로 들어가지 않고, 먼저 목표 슬롯 앞 중앙 레인 위치까지 이동

- 목표 슬롯의 center_x 기준으로 접근 위치 계산
- 중앙 레인 y=0을 따라 이동
- 차량이 슬롯 앞에 도달하면 정지
- 이후 회전 단계로 전환

목표 슬롯 수신
↓
슬롯 방향 및 목표 yaw 계산
↓
LANE_DRIVE: 슬롯 앞 레인 위치로 이동
↓
TURN_TO_SLOT: 슬롯 방향으로 회전
↓
X_ALIGN: 슬롯 중심 x좌표 정렬
↓
PARK_REVERSE: 음수 linear.x로 후진
↓
ROI + 방지턱 조건 확인
↓
DONE: 정지

직접 /ego/cmd_vel 제어 기반 정밀 후진 주차

목표 슬롯 앞까지 접근한 뒤, Nav2가 아니라 parking_node가 직접 차량 속도 명령을 발행해 주차를 마무리하는 과정
정밀한 회전, 정렬, 후진 동작이 필요하기 때문에 /ego/cmd_vel을 직접 제어

3. TURN_TO_SLOT: 슬롯 방향으로 회전

차량이 목표 슬롯 앞에 도착하면, 후진 주차를 위해 슬롯 반대 방향을 바라보도록 회전

- 목표 yaw와 현재 yaw의 오차 계산
- 오차가 작아질 때까지 각속도 명령 발행
- 목표 방향에 도달하면 정지
- 다음 단계인 X_ALIGN으로 전환

4. X_ALIGN: 슬롯 중심 x좌표 정렬

회전 후 차량의 x좌표가 슬롯 중심과 충분히 맞는지 확인

- 목표 슬롯 중심 x좌표와 현재 차량 x좌표 비교
- yaw 오차가 남아 있으면 먼저 yaw 보정
- x 오차가 허용 범위 안이면 후진 주차로 진행
- x 오차가 크면 다시 레인 주행 단계로 돌아가 재정렬

목표 슬롯 수신
↓
슬롯 방향 및 목표 yaw 계산
↓
LANE_DRIVE: 슬롯 앞 레인 위치로 이동
↓
TURN_TO_SLOT: 슬롯 방향으로 회전
↓
X_ALIGN: 슬롯 중심 x좌표 정렬
↓
PARK_REVERSE: 음수 linear.x로 후진
↓
ROI + 방지턱 조건 확인
↓
DONE: 정지

직접 /ego/cmd_vel 제어 기반 정밀 후진 주차

목표 슬롯 앞까지 접근한 뒤, Nav2가 아니라 parking_node가 직접 차량 속도 명령을 발행해 주차를 마무리하는 과정
정밀한 회전, 정렬, 후진 동작이 필요하기 때문에 /ego/cmd_vel을 직접 제어

5. PARK_REVERSE: 후진 주차

정렬이 완료되면 차량을 후진시켜 슬롯 내부로 진입

- /ego/cmd_vel의 linear.x를 음수로 발행
- 후진 중 yaw 오차를 계속 보정
- x 오차에 따라 angular.z 보정
- 차량이 슬롯 ROI 안에 있는지 확인
- 방지턱 접촉 위치 또는 LiDAR 조건을 만족하면 주차 완료

6. 장애물 및 안전 조건 확인

후진 중에는 LiDAR를 이용해 충돌 가능성을 확인

- 후방 장애물 거리 확인
- 좌우 측면 공간 확인
- 인접 주차 차량과의 충돌 위험 확인
- 예상치 못한 후방 장애물이 가까우면 정지
- 측면 장애물이 감지되면 해당 슬롯을 포기하고 재탐색

목표 슬롯 수신
↓
슬롯 방향 및 목표 yaw 계산
↓
LANE_DRIVE: 슬롯 앞 레인 위치로 이동
↓
TURN_TO_SLOT: 슬롯 방향으로 회전
↓
X_ALIGN: 슬롯 중심 x좌표 정렬
↓
PARK_REVERSE: 음수 linear.x로 후진
↓
ROI + 방지턱 조건 확인
↓
DONE: 정지

직접 /ego/cmd_vel 제어 기반 정밀 후진 주차

목표 슬롯 앞까지 접근한 뒤, Nav2가 아니라 parking_node가 직접 차량 속도 명령을 발행해 주차를 마무리하는 과정
정밀한 회전, 정렬, 후진 동작이 필요하기 때문에 /ego/cmd_vel을 직접 제어

7. 주차 완료 판단

주차 완료는 단순히 슬롯 근처에 도착했다고 처리하지 않고, 차량이 슬롯 영역 안에 있고 방지턱 접촉 조건을 만족할 때 완료로 판단

- 차량이 슬롯 ROI 안에 있는지 확인
- 방지턱 y 위치에 도달했는지 확인
- LiDAR로 방지턱 접촉이 감지되는지 확인
- 조건 만족 시 DONE 상태로 전환
- /ego/cmd_vel을 0으로 발행해 차량 정지

목표 슬롯 수신
↓
슬롯 방향 및 목표 yaw 계산
↓
LANE_DRIVE: 슬롯 앞 레인 위치로 이동
↓
TURN_TO_SLOT: 슬롯 방향으로 회전
↓
X_ALIGN: 슬롯 중심 x좌표 정렬
↓
PARK_REVERSE: 음수 linear.x로 후진
↓
ROI + 방지턱 조건 확인
↓
DONE: 정지

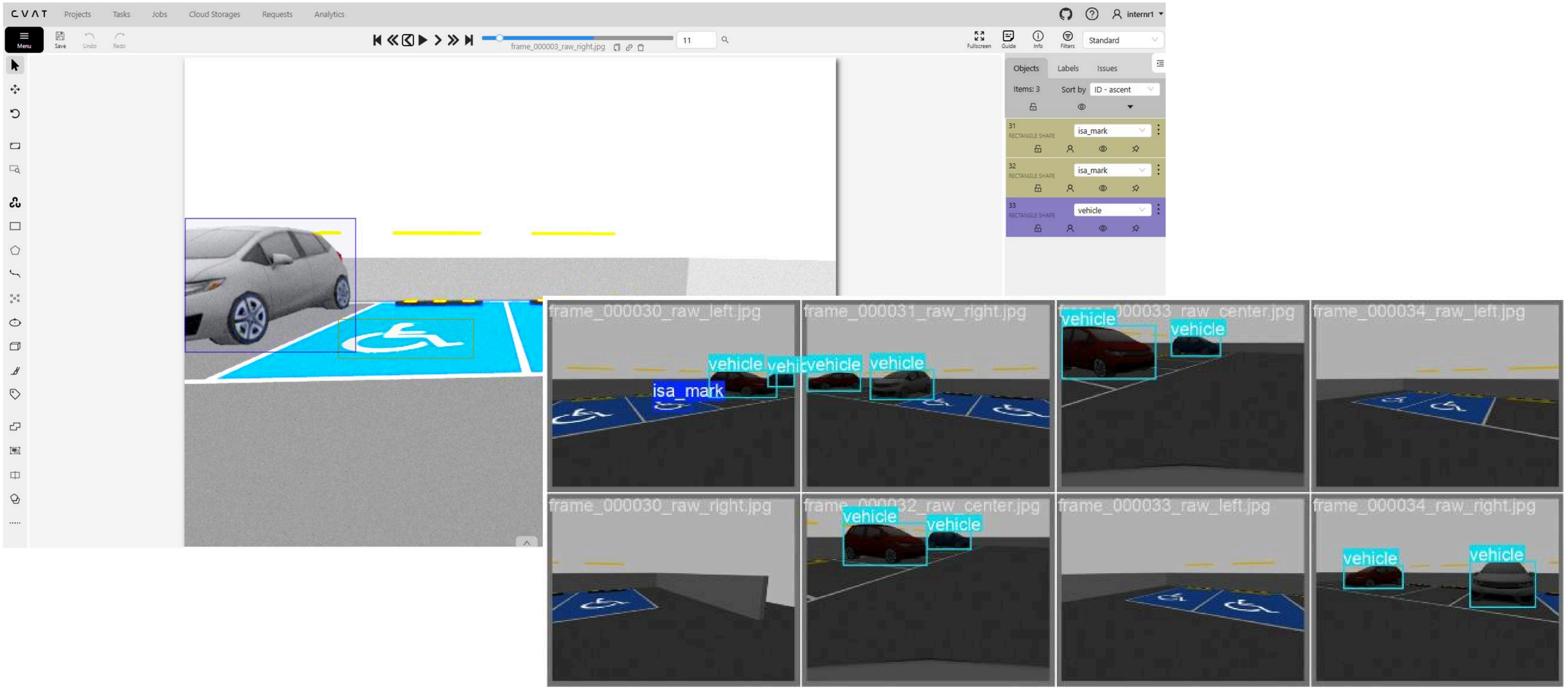
04 결과 및 데모

- 일반/장애인 자격에 따라 서로 다른 슬롯이 선택되는 것을 확인.
- 진입 → 차선 주행 → 회전 → 후진 주차의 전 과정이 상태 머신으로 자동 완료됨.
- 주행 중 전방에 주차 차량/장애물이 있을 때 좌우로 회피 후 목표 슬롯에 도달.
- 주차 완료 시 목표 슬롯이 파란색으로 하이라이트되고, `/parking/result`로 성공 여부·소요 시간·최종 좌표(JSON)가 발행됨.
- RViz에서 슬롯 점유 상태(빨강=점유, 초록=빈칸)와 LiDAR scan, 목표 마커를 실시간 확인.



05 어려움을 겪은 구간

YOLO 도입



05 어려움을 겪은 구간

YOLO 도입

```
50 epochs completed in 0.044 hours.
Optimizer stripped from /home/mj/workspace/Baymax/runs/parking_yolo_ver2-2/weights/last.pt, 6.2MB
Optimizer stripped from /home/mj/workspace/Baymax/runs/parking_yolo_ver2-2/weights/best.pt, 6.2MB

Validating /home/mj/workspace/Baymax/runs/parking_yolo_ver2-2/weights/best.pt...
Ultralytics 8.4.41 🚀 Python-3.10.12 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 3070 Ti Laptop GPU, 7840MiB)
Model summary (fused): 73 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 2/2 4.4it/s 0.5s
                   all         45         69      0.746      0.857      0.832      0.677
              isa_mark         11         14      0.494      0.714      0.669      0.439
               vehicle         35         55      0.998          1      0.995      0.916
Speed: 0.2ms preprocess, 1.7ms inference, 0.0ms loss, 4.4ms postprocess per image
Results saved to /home/mj/workspace/Baymax/runs/parking_yolo_ver2-2
💡 Learn more at https://docs.ultralytics.com/modes/train
VS Code: view Ultralytics VS Code Extension ⚡  at https://docs.ultralytics.com/integrations/vscode
```

데이터셋 규모의 한계

- 추가 데이터셋을 제작하고 파인튜닝을 진행했지만, 장애인 마크는 차량에 비해 학습 데이터가 부족하고 시점·거리·크기 변화에 민감했음

클래스별 성능 차이

- YOLOv8 성능 평가에서 vehicle 클래스는 mAP50이 약 0.995로 높은 성능을 보였지만,
- isa_mark 클래스는 이전보다 성능 저하가 발생함
- 즉, 차량은 안정적으로 검출되지만 장애인 마크는 일관되게 인식하지 못하는 문제

05 어려움을 겪은 구간

YOLO 도입

마크 객체의 시각적 한계

- 장애인 마크는 차량보다 크기가 작고, 카메라 시점에 따라 왜곡되거나 일부만 보일 수 있음
- 또한 Gazebo 환경에서도 거리, 각도, 해상도에 따라 픽셀 영역이 작아져 YOLO가 안정적으로 검출하기 어렵다

슬롯 타입 판단의 신뢰성 문제

- 장애인 마크 인식 결과를 슬롯 타입 판단에 직접 사용할 경우
  - 마크 미검출 또는 오검출로 인해 일반 슬롯과 장애인 전용 슬롯을 잘못 분류할 위험 존재
  - 이는 사용자 자격 기반 주차 시스템에서 중요한 오류로 이어지게 됨

기존 계획

처음에는 YOLOv8을 이용해 주차 슬롯의 점유 여부와 슬롯 유형을 모두 인식하는 방식으로 계획하였으나. (구체적으로는 카메라 이미지에서 두 가지 객체를 검출하려고 했다.)

- vehicle: 슬롯에 차량이 있는지 판단
- isa_mark: 장애인 전용 주차구역인지 판단

구현 방향의 변경

슬롯 타입은 YOLO 인식 결과가 아니라 slot_metadata.yaml에 정의된 정적 메타데이터를 기준으로 판단하도록 변경

YOLOv8은 상대적으로 성능이 높은 vehicle 검출에 집중하여 슬롯 점유 여부 판단에 사용

05 어려움을 겪은 구간

YOLO 바운딩박스 → 슬롯 3D 좌표 매칭 시

2D→3D 역투영 문제

- 카메라는 2D 픽셀 좌표를 출력하지만, 슬롯 점유 판정은 3D 월드 좌표 기준임
- 반대로 슬롯의 알려진 3D 위치를 카메라 이미지의 픽셀 좌표로 투영(projection)한 뒤, YOLO가 감지한 바운딩박스 중심과 거리를 비교하는 방식으로 매칭
- 이를 위해 카메라 위치(extrinsic) + 핀홀 모델(intrinsic)을 수동으로 구현했고 (_CAM_CFG), 차량 yaw가 바뀔 때마다 좌표 변환을 다시 계산해야 하는 구조

```python
def _project_slot_to_pixel(self, camera_key, slot):
    """슬롯 중심 3D 좌표를 카메라 픽셀로 투영. 화면 밖이면 None."""
    cfg = _CAM_CFG[camera_key]

    # 카메라 월드 위치 (robot frame offset → world frame)
    cr, sr = math.cos(self.robot_yaw), math.sin(self.robot_yaw)
    px, py, pz = cfg['pos_robot']
    cam_wx = self.robot_x + cr * px - sr * py
    cam_wy = self.robot_y + sr * px + cr * py
    cam_wz = pz

    # 카메라→슬롯 벡터 (world frame, 슬롯은 z=0 지면)
    dx = slot['center_x'] - cam_wx
    dy = slot['center_y'] - cam_wy
    dz = -cam_wz

    # 카메라 기저 벡터 (world frame)
    ty = self.robot_yaw + cfg['cam_yaw']
    p  = cfg['pitch']
    fwd   = ( math.cos(ty) * math.cos(p),  math.sin(ty) * math.cos(p), -math.sin(p))
    right = ( math.sin(ty),               -math.cos(ty),                0.0)
    down  = (-math.cos(ty) * math.sin(p), -math.sin(ty) * math.sin(p), -math.cos(p))

    depth = dx * fwd[0] + dy * fwd[1] + dz * fwd[2]
    if depth <= 0.3:
        return None

    f = (_IMG_W / 2) / math.tan(cfg['hfov'] / 2)
    u = int(_IMG_W / 2 + f * (dx * right[0] + dy * right[1] + dz * right[2]) / depth)
    v = int(_IMG_H / 2 + f * (dx * down[0]  + dy * down[1]  + dz * down[2])  / depth)

    return (u, v) if (0 <= u < _IMG_W and 0 <= v < _IMG_H) else None
```

05 어려움을 겪은 구간

hatchback 모델을 이용하여 주행 가능한 차체 제작 시

원본 모델

- <static>true</static> — 물리 연산 없이 완전 고정된 장식물
- link 1개, joint 없음, plugin 없음
- .obj mesh를 collision/visual로 쓰는 정적 장애물 모델

주행할 수 있는 자아 차량(ego) 모델 제작

- ego_hatchback/model.sdf에 바퀴 link, revolute joint, diff_drive plugin을 직접 붙여서 주행 가능하게 한 것.

05 어려움을 겪은 구간

hatchback 모델을 이용하여 주행 가능한 차체 제작 시

ODE 물리 엔진의 chassis-wheel 충돌 겹침 문제

- Gazebo는 내부적으로 ODE(Open Dynamics Engine)를 사용해 강체(rigid body) 간 충돌과 관절 물리를 계산합니다.
- ODE의 동작 원칙
  - 두 collision body가 공간적으로 겹쳐있으면 안 된다는 것
  - 겹침이 감지되면 ODE는 두 물체를 밀어내는 반발력(penetration force)을 생성해 분리시키려 함
- 문제는 자동차 모델에서 발생함
  - chassis link의 collision box를 차체 실제 크기(폭 약 1.8m)로 설정하면, 바퀴 link의 collision cylinder(y=±0.85 위치)와 공간적으로 겹치는 현상이 발생.
  - joint가 두 링크를 연결하고 있어도, ODE는 joint의 의도를 무시하고 겹침 자체를 물리적 오류로 판단
    - 결과적으로, 시뮬레이션 시작 순간 바퀴와 chassis 사이에 폭발적인 반발력이 발생하며 차체가 튀어오르거나 회전하는 현상
- 해결 방법은 collision geometry를 실제 시각적 형상과 분리하는 것
  - visual은 원래 차체 크기를 유지하되, collision box 폭만 1.4m로 좁혀 바퀴 inner edge(y=±0.75)와 겹치지 않도록 설계
  - 즉 물리 연산에 쓰이는 형상과 눈에 보이는 형상을 의도적으로 다르게 두어 해결

루프 LiDAR 사각지대

- LiDAR를 차 지붕(z=1.65m)에 하나만 달았을 때 전방 범퍼 근처 저고도 장애물을 감지할 수 없었음
- 높은 센서는 가까운 물체를 내려다봐서 놓치는 구조적 문제로, 전방 범퍼(z=0.5m)에 별도 LiDAR를 추가해 보완

05 어려움을 겪은 구간

hatchback 모델을 이용하여 주행 가능한 차체 제작 시

구동 방식과 주차 알고리즘 설계

- 홀로노믹(holonomic) 로봇은 현재 자세에서 임의의 방향으로 즉시 이동이 가능함
  - 예를 들어 옴니휠 로봇은 앞, 옆, 대각선 어느 방향으로도 즉시 주행 가능
- 반면 일반 자동차는 비홀로노믹(non-holonomic) 시스템
  - 바퀴가 굴러가는 방향으로만 이동할 수 있고, 바퀴 축의 수직 방향(측면)으로는 미끄러지지 않는다는 구속 조건 존재
  - 즉 앞뒤로는 이동할 수 있지만 옆으로는 즉시 이동할 수 없음
  - 주차할 때 한 번에 못 들어가고 여러 번 핸들을 꺾어야 하는 이유

- 이번 모델에서는 실제 자동차의 Ackermann 조향(앞바퀴가 꺾이며 회전)을 구현하는 대신, Gazebo의 `planar_move` 플러그인(`libgazebo_ros_planar_move.so`)을 사용함
  - planar_move는 `linear.x`(전후)·`linear.y`(측면)·`angular.z`(회전)를 모두 직접 받는 홀로노믹 컨트롤러로, 차량이 옆으로도 즉시 이동 가능
- 해당 선택은 주차 알고리즘 전체 설계에 영향
  - 비홀로노믹 차량처럼 핸들을 꺾으며 호를 그리는 경로계획이 필요 없으므로, 슬롯 앞에서 제자리 회전으로 방향을 먼저 맞추고(TURN_TO_SLOT) 직선 후진(PARK_REVERSE)하는 단순화된 시퀀스로 설계
- 후진 중 발생하는 x·yaw 오차는 `linear.y` 측면 보정으로 동시에 잡음
  - 측면 이동이 가능하다는 점이 차선 주행 중 횡보정과 후진 주차 정렬을 모두 가볍게 만든 핵심 전제

05 어려움을 겪은 구간

비동기 멀티노드 시스템의 동시성 문제

ROS2 pub/sub 구조에서 발생하는 타이밍 불일치
- vision_node → decision_node → parking_node로 이어지는 파이프라인 구조
- 각 노드는 독립적으로 실행되며 토픽을 통해 느슨하게 연결

노드 간 파이프라인 지연

- vision_node는 1초 주기로 슬롯 상태를 발행 → decision_node가 이를 받아 처리한 뒤 target_slot을 발행
- parking_node가 이 토픽을 수신하기까지 파이프라인 전체를 통과해야 하므로 최악의 경우 수 초의 latency가 누적
- 이 지연은 예측 불가능하기 때문에 parking_node는 슬롯을 언제까지 기다릴지 명시적으로 SLOT_WAIT_TIMEOUT을 두고 timeout 처리를 설계해야 했습니다.

콜백과 State Machine의 충돌

- parking_node 내부에서도 같은 문제가 발생
- 상태 전환을 담당하는 _tick()은 0.05초마다 실행되지만, 외부에서 들어오는 콜백(_target_cb, _scan_cb 등)은 자신의 주기에 따라 독립적으로 실행
- 예를 들어 parking_node가 이미 LANE_DRIVE 상태로 진입해 슬롯을 향해 주행 중인데, 네트워크 지연으로 늦게 도착한 target_slot 메시지가 현재 슬롯을 덮어쓰면 주행 중 목표 좌표가 바뀌는 문제가 발생함
- 이를 막기 위해 _RECEPTIVE_STATES를 정의해 특정 상태에서만 외부 메시지를 수락하도록 제어권을 명시적으로 구분했습니다.

공통적인 원인

- ROS2의 pub/sub 모델이 메시지 도착 시점을 보장하지 않는다는 점
- 노드가 여러 개로 분산될수록 각 노드의 내부 상태와 외부에서 들어오는 메시지 사이의 타이밍을 명시적으로 설계해야 하고, 이를 소홀히 하면 재현하기 어려운 타이밍 버그로 연결되었음

06 개선 방향

슬롯 위치의 하드코딩 의존성 제거

- 현재: 슬롯 위치와 크기가 slot_metadata.yaml에 수동으로 정의됨
  - 주차장 구조가 바뀌면 YAML을 직접 수정해야 하고, 처음 배포 시 좌표를 일일이 측정해야 함
- 개선: slot_detector_node가 이미 OccupancyGrid 기반으로 슬롯을 동적으로 감지하는 구조를 갖추고 있으나,
  - 메인 파이프라인(vision_node)은 여전히 YAML 좌표를 기준으로 동작 중 → 두 접근을 통합해 지도 기반으로 슬롯 위치 자체를 자동 추출하여 새 환경에 즉시 적용 가능하도록 변경

점유 판정 로직의 안정성 확보

- 현재: YOLO와 LiDAR 중 하나라도 점유를 감지하면 해당 슬롯을 점유로 표시하는 OR 방식
  - 각 센서의 신뢰도 차이나 거리에 따른 정확도 차이가 반영되지 않음
- 개선: 각 센서 관측에 가중치(confidence)를 부여하고 Bayesian 업데이트로 점유 확률을 관리하면 오감지를 개선할 수 있을 것이라고 기대
  - YOLO의 detection confidence score를 이용하여 활용 가능

반응형 제어에서 경로 계획으로

- 현재: 주차 시퀀스가 LANE_DRIVE → TURN_TO_SLOT → PARK_REVERSE의 단순 직선 이동으로 구성
  - 슬롯 앞에서 제자리 회전 후 직선 후진하는 방식이라 좁은 공간에서 다른 차량이나 장애물이 있을 때 취약하다는 특징이 있음
- 개선: 현재 위치에서 슬롯 진입 자세까지의 경로를 사전에 계획하고 추종하는 구조로 전환하여, 장애물 회피와 주차를 통합적으로 처리하도록 개선
  - diff_drive 제약 안에서도 ROS2의 Nav2 planner를 활용하는 방향으로 개선

감사합니다

Team 베이맥스



김민정

인공지능학과 23학번

YOLO 중심의 vision 파이프라인 개발



이윤지

인공지능학과 23학번

맵 제작 및 주차 로직 파이프라인 개발

Appendix. 자세한 노드 구성과 토픽

Gazebo / ego_hatchback

시뮬레이션 차량과 센서를 제공하고, 속도 명령을 받아 차량을 움직인다.

/ego/camera/image_raw 전방 카메라 이미지 발행

/ego/camera_left/image_raw 좌측 카메라 이미지 발행

/ego/scan 360도 LiDAR 데이터 발행

/ego/scan_front 전방 범퍼 LiDAR 데이터 발행

/ego/scan_left 좌측 LiDAR 데이터 발행

/ego/scan_right 우측 LiDAR 데이터 발행

/ego/odom 차량의 현재 위치와 자세 발행

/ego/cmd_vel 차량이 구독하는 속도 명령

Appendix. 자세한 노드 구성과 토픽

parking_node

인식 결과와 슬롯 선택 결과를 바탕으로 “어디로 갈지”와 “어떻게 주차할지”를 실행하는 노드이며,
Nav2 기반 자율 주행과 직접 속도 제어에 기반한 정밀 주차를 연결하는 핵심 컨트롤러이다.

/ego/odom 차량의 현재 위치와 자세를 받아 상태 전환, 목표 도착 여부, 정렬 상태, 주차 완료 여부를 판단한다.

/ego/scan LiDAR 거리 데이터를 받아 주변 장애물과 방지턱 위치를 확인하고 후진 주차 완료 판단에 활용한다.

/parking/slot_states 슬롯별 ID, 위치, 유형, 점유 여부를 받아 목표 슬롯 정보와 주차 가능 상태를 갱신한다.

/parking/target_slot decision_node가 선택한 목표 슬롯 ID를 받아 주차 대상 슬롯으로 설정한다.

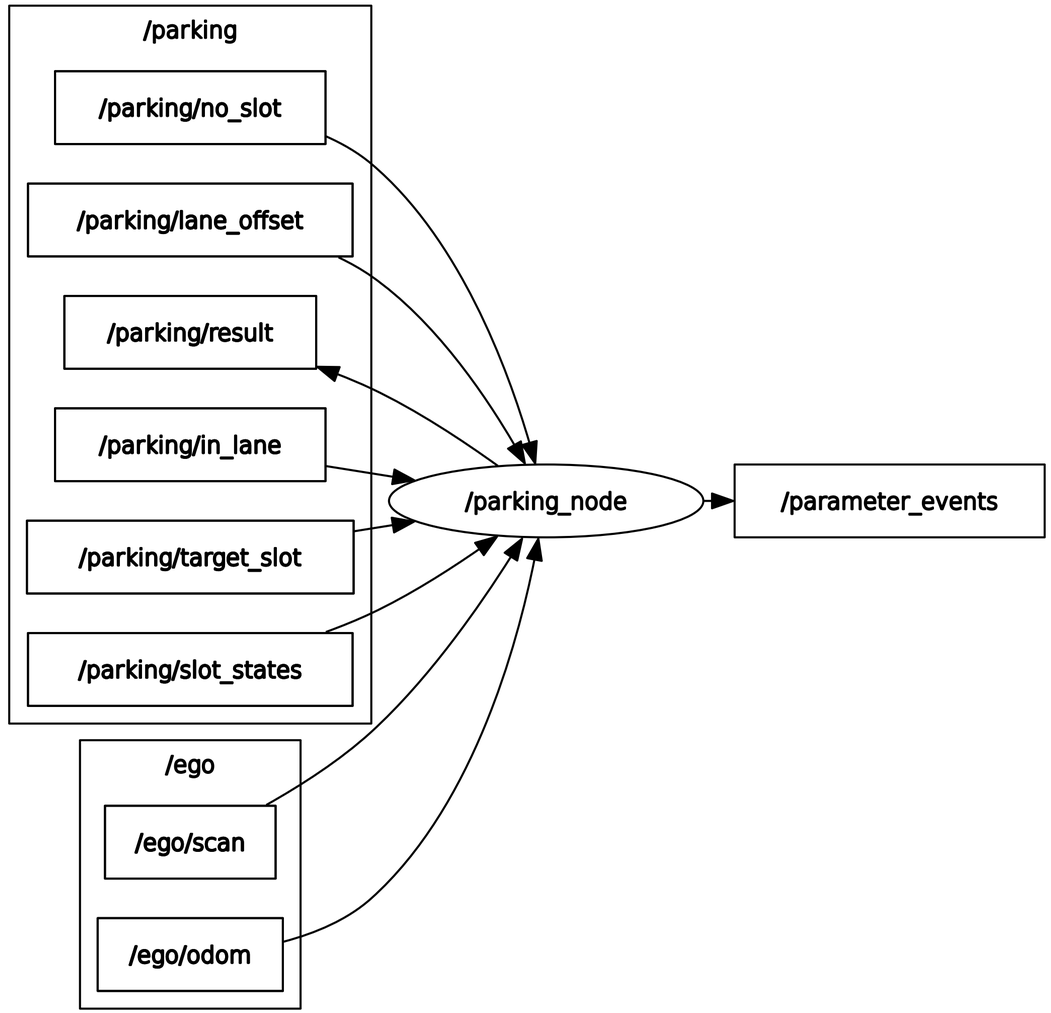
/parking/no_slot 조건에 맞는 빈 슬롯이 없다는 신호를 받아 다음 관측 지점으로 이동하거나 재탐색하도록 한다.

/parking/lane_offset 차선 중심 대비 차량의 좌우 오프셋을 받아 레인 주행 및 정렬 보정에 활용한다.

/ego/cmd_vel 차량 전진, 회전, 정렬, 후진 주차 속도 명령 발행

/parking/result 주차 성공/실패 결과 발행

/parking/target_marker RViz 목표 슬롯 시각화 마커 발행



Appendix. 자세한 노드 구성과 토픽

vision_node

좌우/전방 카메라 이미지 수신
LiDAR 데이터 수신
YOLOv8 모델이 있을 경우 차량 감지 수행
slot_metadata.yaml을 기준으로 슬롯 위치와 유형 확인
슬롯별 점유 여부와 타입을 JSON 형태로 /parking/slot_states에 발행

/ego/odom 차량의 현재 위치와 자세를 받아 상태 전환, 목표 도착 여부, 정렬 상태, 주차 완료 여부를 판단한다.

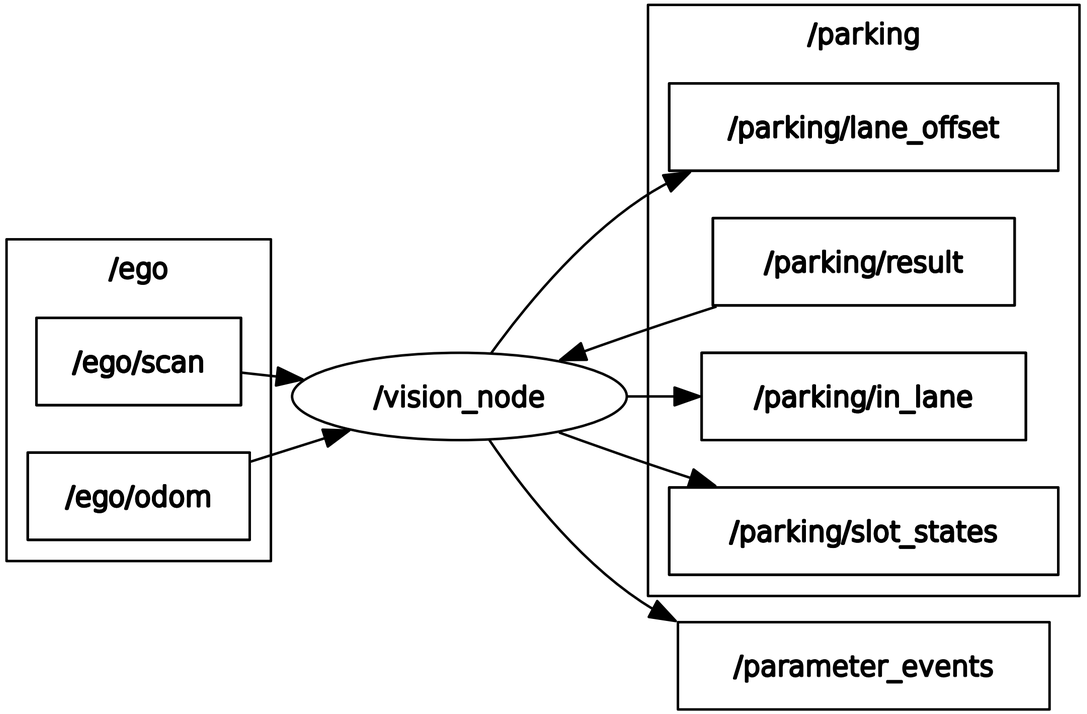
/ego/scan LiDAR 거리 데이터를 받아 주변 장애물과 방지턱 위치를 확인하고 후진 주차 완료 판단에 활용한다.

/parking/slot_states 슬롯별 ID, 위치, 유형, 점유 여부를 받아 목표 슬롯 정보와 주차 가능 상태를 갱신한다.

/parking/slot_markers RViz 시각화용 슬롯 마커 발행

/parking/in_lane 차선 감지 성공 여부 발행

/parking/lane_offset 차선 중심 대비 차량의 좌우 오프셋을 받아 레인 주행 및 정렬 보정에 활용한다.



Appendix. 자세한 노드 구성과 토픽

decision_node

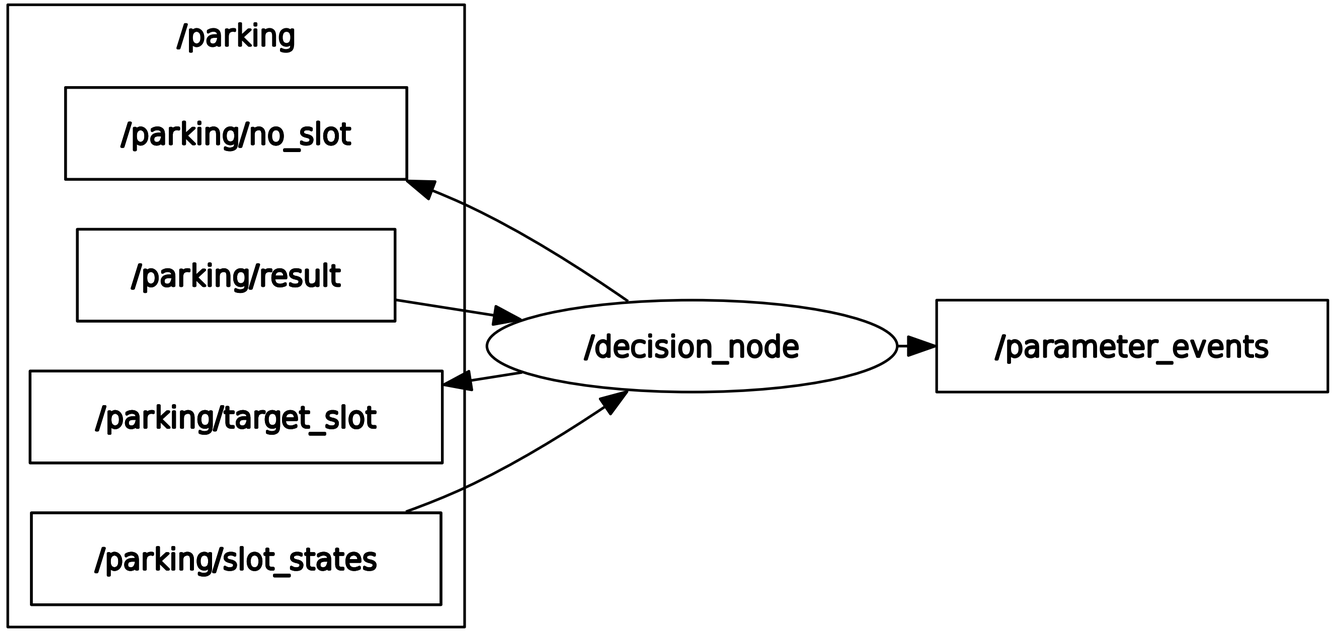
주차 슬롯의 상태를 판단하는 인식 노드

/parking/slot_states sub) vision_node가 판단한 슬롯 상태 입력

/parking/target_slot 선택된 목표 슬롯 ID 발행

/parking/no_slot 조건에 맞는 빈 슬롯이 없을 때 발행

user_credential Parameter: general 또는 handicapped



Appendix. 자세한 노드 구성과 토픽

Nav2 Stack

목표 지점 기반 자율 주행을 위한 planner, controller, navigator 기능을 제공

controller_server 경로 추종 속도 명령 계산

planner_server 목표 지점까지의 경로 계획

behavior_server 회전, 대기, 복구 동작 지원

bt_navigator NavigateToPose action 처리

lifecycle_manager_nav Nav2 노드 lifecycle 관리

/cmd_vel Nav2가 계산한 속도 명령 발행

navigate_to_pose 목표 지점 이동 action

/plan 생성된 전역 경로

/local_costmap/costmap 지역 장애물 costmap

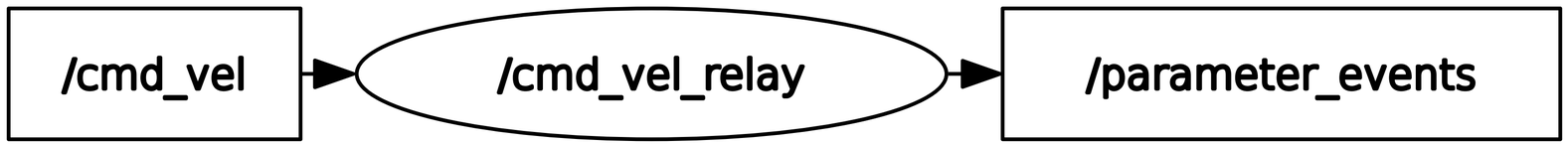
/global_costmap/costmap 전역 costmap

Appendix. 자세한 노드 구성과 토픽

cmd_vel_relay

Nav2가 만든 속도 명령을 Gazebo 차량이 실제로 받는 토픽으로 전달하는 중계 노드

parking_node의 직접 제어와, Nav2의 일반 주행을 구분하기 위하여 도입함
Nav2가 내보내는 속도 명령 토픽과 Gazebo 차량이 실제로 듣는 토픽 이름이 다르게 설정되어있으며,
/cmd_vel로 들어온 명령을 /ego/cmd_vel로 연결함



/cmd_vel Nav2가 계산한 주행 속도 명령을 받음

/ego/cmd_vel Gazebo 차량이 움직일 수 있도록 속도 명령을 전달함